Machine Learning Model Implementation Report

Green Grid Allocator — Smart-Grid Demand and Renewable Forecasting

Traditional machine learning on tabular hourly time-series data

Production demand model	Gradient Boosting
Solar / wind models	HistGradientBoosting / Random Forest
Forecast target	Zone 1 Power Consumption (kW (converted to MW by the simulator))
Dataset	Tetuan City power consumption.csv — 8736 hourly rows
Features	41 engineered features
Held-out test R^2	0.9232
Held-out test RMSE	1845.5 kW
Accuracy within ±10%	90.05 %
Model trained (UTC)	2026-08-12T14:21:59Z
Benchmark run (UTC)	2026-08-12T14:23:51Z
Artifact version	2.0-traditional-ml

Scope statement. This project uses traditional machine learning only. No deep-learning models (CNN, RNN, LSTM, GRU, transformers/BERT) and no image feature extractors (such as SIFT) are used anywhere, because the input data are numeric tabular time series rather than images or natural language. Section 5 explains this in detail.

1. Project Purpose and Forecasting Targets

The Green Grid Allocator is an interactive smart-grid dispatch simulator. Each simulated hour it forecasts how much electricity the city will consume and how much renewable output is available, then dispatches the configured generation fleet in merit order (renewables first, then battery storage, then fossil plants by cost) to serve that demand with the least CO_2 possible.

Three quantities are predicted by machine learning:

#	Target	Type	Model used	Range / unit
1	Zone 1 Power Consumption	Regression (continuous)	Gradient Boosting	kW
2	Solar capacity factor	Regression (continuous)	HistGradientBoosting	0 – 1 (synthetic)
3	Wind capacity factor	Regression (continuous)	Random Forest	0 – 1 (synthetic)

The simulator converts the demand forecast from kW to MW and uses the two capacity factors to scale the installed solar and wind capacity into available MW.

2. Input Dataset

The primary dataset is **Tetuan City power consumption.csv**: power consumption and weather readings for Tetouan, Morocco, recorded every 10 minutes throughout 2017. After hourly resampling it provides **8736 rows** covering 2017-01-08 00:00:00 to 2017-12-30 23:00:00.

Column	Meaning	Unit	Role
DateTime	Timestamp of the reading	—	Index
Temperature	Ambient air temperature	°C	Feature
Humidity	Relative humidity	%	Feature
Wind Speed	Wind speed	m/s	Feature + wind target driver
general diffuse flows	Broad diffuse solar radiation	W/m²	Feature + solar target driver
diffuse flows	Diffuse solar radiation	W/m²	Feature + solar target driver
Zone 1 Power Consumption	Distribution zone 1 demand	kW	**Prediction target**
Zone 2 Power Consumption	Distribution zone 2 demand	kW	Lagged history only
Zone 3 Power Consumption	Distribution zone 3 demand	kW	Lagged history only

Zone 2 and Zone 3 readings for the current hour are deliberately excluded from the feature set — only their lagged and rolling history is used, since a live meter reading for the hour being predicted would not be available at forecast time.

The repository also ships *Steel_industry_data.csv*, *smart_grid_stability_augmented.csv* and *household_power_consumption.txt*. These are retained from the original project for reference and are not used by the training, benchmarking or inference code.

Derived target classes.

For the auxiliary classification view (Section 8) the continuous demand is binned into three classes using tertiles of the training partition only:

Class	Definition	Boundary (kW)
Low	demand < lower tertile	28882
Medium	lower tertile ≤ demand < upper tertile	28882 – 35885
High	demand ≥ upper tertile	35885

3. Data Preprocessing

All preprocessing lives in **ml_pipeline.py** and is imported by **train.py**, **test.py** and **grid_sim.py**, so the feature definitions used at training time and at inference time can never drift apart.

- **Column normalisation** — surrounding whitespace is stripped from every CSV header.
- **Timestamp parsing** — `DateTime` is parsed day-first (the file uses D/M/YYYY), set as the index and sorted chronologically.
- **Hourly resampling** — the six 10-minute readings in each hour are averaged to a regular 1-hour grid, which removes high-frequency meter noise and gives a uniform step for the lag features.
- **Missing values** — any hour with no reading is forward-filled (then back-filled at the very start), preserving the temporal order without inventing values from the future.
- **Chronological split** — 70% train / 15% validation / 15% test, cut by time. The data are **never shuffled**.
- **Scaling** — a `StandardScaler` (zero mean, unit variance) is fitted on the training partition only and then applied to validation and test. For SVR and the MLP the target is standardised as well via `TransformedTargetRegressor`, because demand values of ~3×10^4 kW otherwise swamp their default loss scales.

Partition	Rows	Period	Used for
Train	5997	2017-01-08 00:00:00 → 2017-09-14 20:00:00	Fitting every model
Validation	1285	2017-09-14 21:00:00 → 2017-11-07 09:00:00	Model selection + ensemble weights
Test	1286	2017-11-07 10:00:00 → 2017-12-30 23:00:00	Final reported metrics only

Because the split is chronological, the test period is a genuine future window relative to everything the models saw. This is stricter — and far more realistic for forecasting — than a random split, which would let a model interpolate between neighbouring hours it had already memorised.

4. Feature Engineering

The demand model consumes **41 features** in four families. Every one is computed by `ml_pipeline.build_demand_frame()`.

Family	Features	Count	Why it helps
Weather (current hour)	Temperature, Humidity, Wind Speed, general diffuse flows, diffuse flows	5	Air-conditioning and lighting load track temperature and daylight.
Calendar + cyclical time	hour, dow, month and sin/cos pairs for each	9	Encodes daily, weekly and seasonal rhythms; sin/cos keeps hour 23 adjacent to hour 0 instead of maximally distant.
Power lags & rolling means	For each of Zone 1/2/3: lag 24, lag 168, rolling mean 24, rolling mean 168	12	Yesterday's and last week's demand at the same hour are the single strongest predictors; rolling means capture the recent level.
Weather lags & rolling means	For each weather channel: lag 24, lag 168, rolling mean 24	15	Thermal inertia — buildings respond to the last day of weather, not only the current instant.

Leakage prevention.

Every historical feature used to predict hour t is built only from information available strictly before t:

- Lag features use `shift(24)` and `shift(168)`, exposing the value from exactly one day and one week earlier.
- Rolling means are computed and then `shift(1)`-ed, so the window closes at $t-1$ and never includes the current hour.
- The raw Zone 1/2/3 readings at hour t are excluded from the feature list entirely.
- Scalers and class thresholds are fitted on training rows only.
- Current-hour weather *is* included, which is standard practice: at forecast time the weather for the target hour is a known exogenous input, supplied by a meteorological forecast.

At inference time `grid_sim.py` rebuilds exactly these features from the real historical series for the simulator's current hour, using the same window semantics, and reads the ordered feature list from `models/features.json` so the vector is assembled in the order the model was trained on.

5. Why SIFT and Deep Learning Are Not Used

SIFT (Scale-Invariant Feature Transform) is not used in this project, and it would not be meaningful to use it. SIFT is a computer-vision algorithm that locates keypoints in a 2-D image and describes the gradient patterns around them. It presupposes pixels, spatial neighbourhoods and scale space. The inputs here are numeric readings from meters and weather sensors arranged as rows in a table — there is no image, no pixel grid and no spatial structure for a keypoint detector to operate on. The same applies to any CNN-based image feature extraction.

For the same category reason, this project also does not use text transformers. BERT and related models operate on tokenised natural language; a sequence of kilowatt readings is not text, so reporting a BERT score here would be meaningless. The benchmark therefore prints an explicit *not applicable* row for these families rather than fabricating numbers for them.

Deep sequence models (RNN, LSTM, GRU) *are* technically applicable to time series, but they are outside this project's scope: the requirement is a traditional machine-learning implementation. No

TensorFlow or PyTorch dependency is present. The feature engineering in Section 4 is what supplies the temporal context instead — the lag and rolling-window features hand the model an explicit summary of the previous day and week, which is the role a recurrent layer would otherwise play.

The actual feature extraction used is therefore: time-derived cyclical encodings, lag values, rolling statistics, and weather variables — all tabular, all numeric.

6. Algorithms Benchmarked

Every model below is trained on the same training partition and scored on the same held-out test period. All are traditional machine-learning algorithms from scikit-learn, plus XGBoost.

Algorithm	Family	Role and suitability for this task
Linear Regression	Linear	Ordinary least squares. Transparent baseline; underfits because load responds non-linearly to temperature and hour.
Ridge Regression	Linear (L2)	Adds L2 shrinkage, which stabilises coefficients when the lag and rolling features are strongly correlated with each other.
Lasso Regression	Linear (L1)	L1 penalty drives weak coefficients to zero, giving implicit feature selection over the 41 inputs.
ElasticNet	Linear (L1+L2)	Blends both penalties; handles correlated feature groups better than pure Lasso.
KNN Regressor	Instance-based	Averages the nearest historical hours in feature space. Intuitive, but weakens in high dimensions.
Decision Tree	Tree	Single interpretable tree; captures non-linear thresholds but has high variance on its own.
Random Forest	Bagged trees	Averages many decorrelated deep trees. Robust, handles feature interactions, needs little tuning.
Extra Trees	Bagged trees	Like Random Forest but with randomised split thresholds — lower variance and noticeably faster to fit.
Gradient Boosting	Boosted trees	Fits trees sequentially on residuals. Strong on structured tabular data; slower to train.
HistGradientBoosting	Boosted trees	Histogram-binned boosting — near-identical accuracy to classic boosting at a fraction of the training time.
AdaBoost	Boosted trees	Re-weights hard examples across rounds, using depth-8 trees as base learners here.
Support Vector Regressor	Kernel	RBF-kernel epsilon-tube regression. Applicable, but scales poorly with sample count and is sensitive to feature/target scaling.
MLP Regressor	Shallow neural net	scikit-learn's fully-connected network. Included as the traditional ML upper bound; it is not a deep-learning framework model.
XGBoost	Boosted trees	Regularised gradient boosting; a standard strong baseline for tabular regression. Optional — the benchmark skips it cleanly if not installed.
Gaussian Naive Bayes	Probabilistic classifier	**Classification baseline only.** Predicts a Low/Medium/High demand class, never a continuous value — see Section 7.

Explicitly not applicable.

Method	Why it is not applicable here
BERT	Text/NLP transformer - the inputs are numeric tabular time series, not tokenised language.
CNN	Convolutional image model - there are no images or spatial grids in this dataset.
RNN / LSTM / GRU	Deep sequence models excluded by project scope; only traditional ML is used here.
SIFT	Image keypoint descriptor - meaningless for numeric weather/power readings.

7. Regression Models vs. the Naive Bayes Baseline

This distinction matters because it is a common source of misreported results. Demand forecasting is a **regression** problem: the answer is a real number of kilowatts. Naive Bayes is a **classifier**: it outputs a discrete label and a class probability, and it has no mechanism for producing a continuous kilowatt value.

Therefore GaussianNB is never treated as a demand regressor in this project. It is trained on the derived Low/Medium/High labels and reported separately, with MAE, RMSE and R^2 left blank because those quantities are undefined for it. Conversely, the regression models are never presented as classifiers — their precision, recall and F1 are always labelled as *derived* demand-class metrics obtained by binning their continuous output.

8. Evaluation Metrics

Native regression metrics.

Metric	Formula	Reading
MAE	$(1/n) \cdot \Sigma \lvert y_i - \hat{y}_i \rvert$	Mean absolute error in kW. Same units as demand; robust to outliers.
RMSE	$\sqrt{[\ (1/n) \cdot \Sigma\ (y_i - \hat{y}_i)^2\ ]}$	Root mean squared error in kW. Penalises large misses more heavily.
R^2	$1 - \Sigma(y_i - \hat{y}_i)^2 \ / \ \Sigma(y_i - \bar{y})^2$	Share of demand variance explained. 1.0 is perfect; 0.0 equals always predicting the mean.
Accuracy within ±10%	$100 \cdot (1/n) \cdot \Sigma\ 1[\ \lvert \hat{y}_i - y_i \rvert \le \tau \cdot \lvert y_i \rvert\ ]$	Percentage of hours forecast within ±10% of actual demand, with $\tau = 0.1$. Configurable via --tolerance.

Derived demand-class metrics.

Precision, recall, F1 and classification accuracy do not exist natively for a regression task. They are obtained here through a documented auxiliary view, and are always labelled as derived:

- Compute tertile thresholds on the **training** target only.
- Apply those exact thresholds to both the actual and the predicted test demand, producing Low/Medium/High labels for each.
- Compute **macro-averaged** precision, recall and F1 across the three classes, so each class counts equally regardless of its frequency.

Metric	Formula	Reading
Precision	TP / (TP + FP)	Of the hours predicted to be in a class, the share that truly were.
Recall	TP / (TP + FN)	Of the hours truly in a class, the share that were found.
F1	$2 \cdot (P \cdot R) \ / \ (P + R)$	Harmonic mean of precision and recall.
Class accuracy	correct labels / total labels	Share of test hours placed in the correct demand band.

A model can post a high derived F1 while still having mediocre RMSE — landing in the right third of the demand range is a much easier task than predicting the exact kilowatts. This is precisely why these figures must never be quoted as regression accuracy.

9. Benchmark Results

Produced by `python test.py --quick` on 2026-08-12T14:23:51Z (seed 42), evaluated on the held-out test period 2017-11-07 10:00:00 → 2017-12-30 23:00:00. Derived columns are marked with an asterisk.

#	Model	Type	±10% Acc	MAE	RMSE	R²	Prec*	Rec*	F1*
1	**AdaBoost**	regressor	91.84	1271.3	1751.9	0.9308	0.905	0.898	0.900
2	**Ensemble: Top-3 (AdaBoost, Random Forest, Extra Trees)**	ensemble	91.29	1315.3	1780.4	0.9285	0.907	0.899	0.901
3	Random Forest	regressor	88.72	1407.5	1907.5	0.9180	0.903	0.899	0.900
4	Extra Trees	regressor	88.41	1400.3	1870.1	0.9211	0.906	0.898	0.900
5	Ensemble: Extra Trees + XGBoost	ensemble	86.39	1463.8	1902.8	0.9184	0.897	0.890	0.892
6	Gradient Boosting	regressor	81.18	1793.2	2305.3	0.8802	0.863	0.865	0.863
7	HistGradientBoosting	regressor	81.10	1715.4	2186.6	0.8922	0.887	0.890	0.887
8	XGBoost	regressor	80.17	1674.9	2131.0	0.8976	0.893	0.890	0.889
9	Ensemble: Random Forest + XGBoost	ensemble	80.17	1674.9	2131.0	0.8976	0.893	0.890	0.889
10	Decision Tree	regressor	69.83	2368.6	3086.2	0.7852	0.798	0.784	0.789
11	KNN Regressor	regressor	66.72	2375.3	2922.7	0.8074	0.824	0.835	0.827
12	MLP Regressor	regressor	61.28	2928.7	3860.2	0.6640	0.763	0.780	0.761
13	ElasticNet	regressor	59.80	2593.7	3010.7	0.7956	0.836	0.847	0.838
14	Ridge Regression	regressor	53.19	2806.4	3241.0	0.7632	0.820	0.833	0.822
15	Lasso Regression	regressor	53.19	2806.5	3241.1	0.7631	0.820	0.833	0.822
16	Linear Regression	regressor	53.11	2808.1	3242.8	0.7629	0.820	0.833	0.822
17	Support Vector Regressor	regressor	41.84	3785.6	4452.7	0.5530	0.687	0.647	0.624
—	Gaussian Naive Bayes	classifier baseline	—	—	—	—	0.820	0.831	0.823
—	BERT	not applicable	—	—	—	—	—	—	—
—	CNN	not applicable	—	—	—	—	—	—	—
—	RNN / LSTM / GRU	not applicable	—	—	—	—	—	—	—
—	SIFT	not applicable	—	—	—	—	—	—	—

* Derived demand-class metrics (Section 8), not native regression scores. Rows without a rank do not predict continuous demand and are excluded from ranking: the Naive Bayes classification baseline and the not-applicable method families.

10. Model Combinations (Ensembles)

Three genuine weighted ensembles are built. A weighted ensemble produces its forecast as a convex blend of its members' predictions:

$$\hat{y}_{ens} = w_1 \cdot \hat{y}_1 + w_2 \cdot \hat{y}_2 + \ldots + w_k \cdot \hat{y}_k \quad \text{subject to} \quad w_i \geq 0 \quad \text{and} \quad \Sigma\, w_i = 1$$

The weights are fitted on the **validation** partition only, by minimising validation RMSE over the simplex (SLSQP, with non-negative least squares as a fallback). Constraining weights to be non-negative and to sum to one keeps the blend interpretable and prevents one member's errors being amplified to cancel another's. The resulting weights are then applied unchanged to the untouched test predictions.

Ensembles help when members make *different* mistakes: averaging cancels independent error while preserving the shared signal. When members are highly correlated, the optimiser can legitimately place all weight on the single best member — the benchmark reports that honestly rather than forcing an artificial blend.

Ensemble	Fitted weights (validation)	±10% Acc	RMSE	R²
Ensemble: Top-3 (AdaBoost, Random Forest, Extra Trees)	non-negative weights fitted on the validation partition (simplex-constrained RMSE minimisation): AdaBoost=0.398, Random Forest=0.000, Extra Trees=0.602	91.29	1780.4	0.9285
Ensemble: Extra Trees + XGBoost	non-negative weights fitted on the validation partition (simplex-constrained RMSE minimisation): Extra Trees=0.635, XGBoost=0.365	86.39	1902.8	0.9184
Ensemble: Random Forest + XGBoost	non-negative weights fitted on the validation partition (simplex-constrained RMSE minimisation): Random Forest=0.000, XGBoost=1.000 - note: validation RMSE was minimised by a single member, so this blend collapses onto that model	80.17	2131.0	0.8976

11. How the Best Model Is Selected

All models and ensembles are ranked by a single declared rule: **highest accuracy-within-tolerance, then lowest RMSE, then highest R²**.

- **Primary — accuracy within ±10%.** This is the criterion an operator cares about: how often the forecast is close enough to be actionable.
- **First tie-break — lowest RMSE.** Among models that hit the band equally often, prefer the one whose misses are smaller.
- **Second tie-break — highest R².** Prefer the model explaining more of the variance overall.

Category	Winner	Why
Best individual	AdaBoost	AdaBoost ranks #1 under the rule 'highest accuracy-within-tolerance, then lowest RMSE, then highest R²': it reached 91.84% of forecasts within +/-10% of actual demand, with RMSE 1751.9 kW and R² 0.9308 on the held-out test period.
Best combination	Ensemble: Top-3 (AdaBoost, Random Forest, Extra Trees)	Ensemble: Top-3 (AdaBoost, Random Forest, Extra Trees) ranks #2 under the rule 'highest accuracy-within-tolerance, then lowest RMSE, then highest R²': it reached 91.29% of forecasts within +/-10% of actual demand, with RMSE 1780.4 kW and R² 0.9285 on the held-out test period.

Category	Winner	Why
Production model	Gradient Boosting	Selected by **train.py** using the same rule but scored on the **validation** partition, because choosing a production model on test results would leak the test set and inflate the reported performance. The production choice may therefore differ from the test-set winner — that difference is expected and is the honest outcome of keeping the test period untouched.

Held-out test performance of the deployed model (Gradient Boosting): MAE 1392.1 kW, RMSE 1845.5 kW, R^2 0.9232, accuracy within ±10% 90.05%, derived macro-F1 0.9042.

12. Production Deployment and Outputs

Where each model lives in the running application:

Stage	File	What happens
Training	train.py	Fits every candidate, selects on validation, refits the winner on train+validation, writes artifacts to models/.
Shared pipeline	ml_pipeline.py	Single definition of cleaning, features, splitting, metrics and the model registry.
Benchmark	test.py	Trains and scores the full comparison, writes benchmark_results.json and .csv.
Inference	grid_sim.py	GridSimulator.load_artifacts() loads the production models; predict_demand(), predict_solar_cf() and predict_wind_cf() run each simulated hour.
Web API	app.py	Serves the simulator endpoints plus GET /api/model-performance, which feeds the dashboard's Model Performance panel.

Output formats.

Output	Type	Produced by	Consumed as
Demand forecast	Continuous float, kW	Gradient Boosting regressor	Divided by 1000 → MW; drives the dispatch stack.
Solar capacity factor	Continuous float, 0–1	HistGradientBoosting regressor	Clipped to [0,1]; multiplied by installed solar MW.
Wind capacity factor	Continuous float, 0–1	Random Forest regressor	Clipped to [0,1]; multiplied by installed wind MW.
Low / Medium / High class	Discrete label	Derived binning (and GaussianNB baseline)	Evaluation and reporting only — the simulator itself always uses the continuous forecast.

The demand-class output is a reporting construct. It is not fed into dispatch, because dispatch needs an actual megawatt figure rather than a band.

13. Installation and Execution

1. Install dependencies

```
cd mlpbl
python -m venv .venv
source .venv/bin/activate  # Windows: .venv\Scripts\activate
pip install -r requirements.txt
```

2. Train the production models (writes to models/)

```
python train.py
python train.py --full    # heavier hyper-parameters
python train.py --tolerance 0.05
```

3. Run the benchmark (writes benchmark_results.json / .csv)

```
python test.py
python test.py --quick
python test.py --full
python test.py --tolerance 0.10
```

4. Launch the interfaces

```
python app.py      # web UI → http://127.0.0.1:8000
python grid_sim.py # terminal simulator
```

5. Regenerate this report

```
python docs/generate_report.py
```

Training must run before the app: the simulator raises an actionable error naming the missing artifacts and the exact command to fix it, rather than failing with a raw traceback.

14. Limitations

- **Solar and wind targets are synthetic.** The dataset contains no measured renewable generation. Solar capacity factor is derived from diffuse-flow readings plus a solar-altitude and seasonality model; wind capacity factor comes from a standard turbine power curve applied to wind speed. Their very high R^2 values therefore mainly show that the models learned a deterministic formula — they are *not* evidence of real-world renewable forecast skill, and should not be quoted as such. Only the demand metrics reflect a genuine prediction problem.
- **Classification metrics are derived, not native.** Precision, recall and F1 come from binning a continuous forecast into tertiles. They measure band placement, not kilowatt accuracy, and are easier to score well on.
- **Single city, single year.** The data cover Tetouan in 2017. Seasonal coverage is complete but there is only one instance of each season, so the model cannot distinguish a recurring seasonal pattern from a one-off event that year.
- **The test period is winter.** A chronological split puts November–December in the test set, so reported metrics reflect cold-season load behaviour more than summer cooling peaks.
- **One-step-ahead framing.** The model predicts the current hour given weather and history up to the previous hour. Multi-day-ahead forecasting would require recursive prediction and would show materially worse error.
- **Hardware and runtime.** Default (quick) hyper-parameters are sized to finish in a couple of minutes on a normal laptop CPU. --full improves some models but takes considerably longer; no GPU is used or needed anywhere.
- **Weather is assumed known.** Current-hour weather is treated as an available input. In live operation it would come from a meteorological forecast, whose own error would add to the demand forecast error.

15. Project File Map

Path	Purpose
ml_pipeline.py	Shared preprocessing, features, splits, metrics, model registry
train.py	Trains and saves the production models
test.py	Full benchmark + ensembles → JSON/CSV
grid_sim.py	Simulator core and terminal UI; production inference
app.py	FastAPI server and REST API
static/	Dashboard (index.html, app.js, style.css)
models/	Trained artifacts, metadata and benchmark results
docs/	This report and its generator script
Tetuan City power consumption.csv	Primary dataset